



WEBSITE EVALUATION DOCUMENTATION
Mangosuthu University of Technology
Department of Information and Communication
Technology
(ICT Webpage)

Initials and surname	Student Number
S Mthethwa	22320593
LW Cele	22323721

1. INTRODUCTION	2
REDESIGNED WEBSITE.....	4
2. PROBLEM STATEMENT	5
3. PROJECT OBJECTIVES.....	6
3.1 Main Objective	6
3.2 Specific Objectives	6
4. WEBSITE ANALYSIS AND UXD REDESIGN	7
4.1 Analysis of Existing Website.....	7
4.2 UXD Improvements Implemented	7
4.3 Responsive Design	8
5. FRONTEND DEVELOPMENT.....	8
5.1 HTML Structure	8
5.2 CSS Design and Styling	9
5.3 JavaScript Interactivity	9
6. BACKEND DEVELOPMENT USING NODE.JS AND EXPRESS	10
6.1 Backend Architecture	10
PROJECT FOLDER STRUCTURE	11
6.2 Express Server Configuration.....	11
SERVER RUNNING	12
6.3 Routes and HTTP Methods.....	12
7. DATABASE DESIGN AND INTEGRATION	13
7.1 Database Technology.....	13
DATABASE	13
7.2 Database Design	14
7.3 CRUD Operations	15
Create.....	15
Read	15
Update.....	15
Delete	15

8. MIDDLEWARE IMPLEMENTATION	15
MIDDLEWARE.....	16
9. AI AND SMART FEATURE IMPLEMENTATION.....	17
AI FEATURE.....	18
10. FULL SYSTEM INTEGRATION	18
11. TESTING AND RESULTS	19
12. CHALLENGES FACED.....	20
13. CONCLUSION	20

1. INTRODUCTION

The purpose of this project was to redesign and improve the current ICT Department website into a modern, smart, interactive and database-driven web application. The existing departmental website mainly provided static information and lacked advanced functionality, interactive services and backend integration.

This project aimed to apply both frontend and backend web development concepts learned in Internet Programming 2. The system was developed using HTML, CSS, JavaScript, Node.js, Express and a database system. The website was designed according to User Experience Design (UXD) principles to improve usability, responsiveness, accessibility and overall user satisfaction.

The redesigned system allows users to interact with the website through forms, dynamic content, smart functionality and database-driven services. The website also includes middleware functionality, CRUD operations and an AI-driven smart feature to improve communication and support for students and staff.

The project demonstrates the integration between frontend technologies, backend technologies and database systems to create a complete full-stack web application.



Department of Information & Communication Technology

Why Study ICT?



Every company needs an ICT specialist, who not only understands the technology behind computers and mobile devices but also understands the needs of the company and the workforce. So an ICT professional is in charge of the physical hardware like computers, printers, and copiers, as well as software, communication and connection systems, and online platforms. They also know how to use these technologies to streamline business processes. An ICT specialist designs, maintains, and services the systems used to generate, store, send and retrieve data. The ICT qualification from MUT has been specifically designed to prepare students directly for the workplace.



Diploma in Information Technology

NQF Level: 6

HEQSF Qualification Type: 63

SAQA Credits: 360

SAQA Qualification ID: 96862

CESM Code: 06

Rationale for Qualification

This qualification has been re-aligned to the new HEQSF and takes into account the latest ICT industry requirements. The qualification has also been formulated to reflect the latest workplace-based trends within the ICT industry, to ensure that qualifying learners will have accessibility to employment within the industry. The inclusion of a work integrated learning component as part of a credit-bearing module at NQF-6 is a unique concept within the University of Technology ICT sector in SA. This, together with the project-based and problem-solving approach embedded in the qualification, will have the defined purpose of capacitating the student with workplace-applicable competencies.

REDESIGNED WEBSITE



2. PROBLEM STATEMENT

The original ICT Department website had several limitations that negatively affected the user experience and reduced the usefulness of the platform. The website mainly displayed static content and lacked interactive functionality that modern users expect from web applications.

The website did not provide meaningful engagement between students and the department. Users could not easily submit enquiries, access smart support features or interact dynamically with the system. Navigation was not fully user-friendly and some pages lacked consistency in layout and design.

The existing website also lacked backend functionality and database integration. Information was not dynamically managed, meaning that updates required manual editing of static pages. The website did not demonstrate CRUD operations, middleware processing or intelligent features.

In addition, the website had usability and accessibility challenges, especially on mobile devices. Some sections were difficult to navigate and the overall responsiveness of the website was limited.

This project therefore aimed to solve these problems by developing a smart, interactive and fully integrated departmental web application.

3. PROJECT OBJECTIVES

3.1 Main Objective

The main objective of this project was to redesign the ICT Department website into a smart, interactive and database-driven web application.

3.2 Specific Objectives

The specific objectives of the project were:

- To improve the overall user experience using UXD principles.
- To redesign the website interface using HTML and CSS.
- To implement interactive frontend functionality using JavaScript.
- To develop a backend using Node.js and Express.
- To connect the system to a database.
- To demonstrate CRUD operations.

- To implement middleware for validation, logging and error handling.
- To integrate an AI or smart feature into the website.
- To ensure responsive design across multiple devices.
- To demonstrate complete frontend-backend integration.

4. WEBSITE ANALYSIS AND UXD REDESIGN

4.1 Analysis of Existing Website

The existing ICT Department website was analysed to identify weaknesses and usability problems. Several issues were identified during the analysis phase.

Firstly, the navigation structure was not fully user-friendly. Users experienced difficulty locating important information because menu items were not well organized.

Secondly, the website layout lacked consistency. Different pages used inconsistent spacing, colours and typography, which negatively affected the visual experience.

Thirdly, the website did not support enough interaction between users and the department. There were limited forms, dynamic content areas or feedback mechanisms.

Another challenge was poor responsiveness on smaller screens. The website did not adapt effectively to mobile devices, which affected readability and accessibility.

Accessibility concerns were also identified, including poor text contrast, inconsistent button design and insufficient feedback messages.

4.2 UXD Improvements Implemented

Several UXD improvements were implemented in the redesigned website.

The navigation system was improved to make it easier for users to move between pages and locate information quickly. Consistent navigation menus were added across all pages.

The colour scheme and typography were redesigned to improve readability and visual consistency. Better spacing and alignment were introduced to create a cleaner and more modern layout.

Responsive design principles were applied to ensure that the website functions properly on desktops, tablets and mobile devices.

Interactive elements such as forms, feedback messages, search functionality and dynamic content sections were also added to improve user engagement.

The overall user interface was simplified to reduce confusion and improve accessibility.

4.3 Responsive Design

Responsive web design techniques were used to ensure compatibility across different screen sizes and devices.

CSS Flexbox and Grid layouts were implemented to create adaptive page structures. Media queries were used to adjust layouts, font sizes and navigation components depending on the device size.

The website was tested on desktop, tablet and mobile views to ensure consistent user experience across all devices.

5. FRONTEND DEVELOPMENT

5.1 HTML Structure

The frontend structure of the website was developed using HTML. Multiple pages were created to organize departmental information and services.

The pages included:

- Home page

- About page
- Staff page
- Contact page
- Events page
- FAQ page
- Student support page
- Appointment booking page

Each page was designed with a consistent structure to improve navigation and user experience.

5.2 CSS Design and Styling

CSS was used to improve the visual appearance of the website. The design focused on readability, consistency and responsiveness.

Modern colour combinations were used to create a professional appearance suitable for an academic department website. Typography was carefully selected to improve readability.

Flexbox and Grid layouts were used to organize page content effectively. Buttons, cards and navigation bars were styled consistently throughout the application.

Hover effects, transitions and animations were also implemented to improve user interaction.

5.3 JavaScript Interactivity

JavaScript was used to implement meaningful interactive functionality within the website.

Interactive features included:

- Form validation
- Dynamic menus

- Search filtering
- Modal windows
- Content toggling
- User feedback messages
- Smart chatbot interaction

Form validation was implemented to ensure that users provide valid input before submission. Error messages and success notifications improved user feedback.

Dynamic search and filtering functionality improved content accessibility and helped users find information more efficiently.

The interactive features enhanced the overall user experience by making the website more responsive and user-friendly.

6. BACKEND DEVELOPMENT USING NODE.JS AND EXPRESS

6.1 Backend Architecture

The backend system was developed using Node.js and Express. The backend was responsible for processing requests, handling routes, managing database operations and supporting frontend functionality.

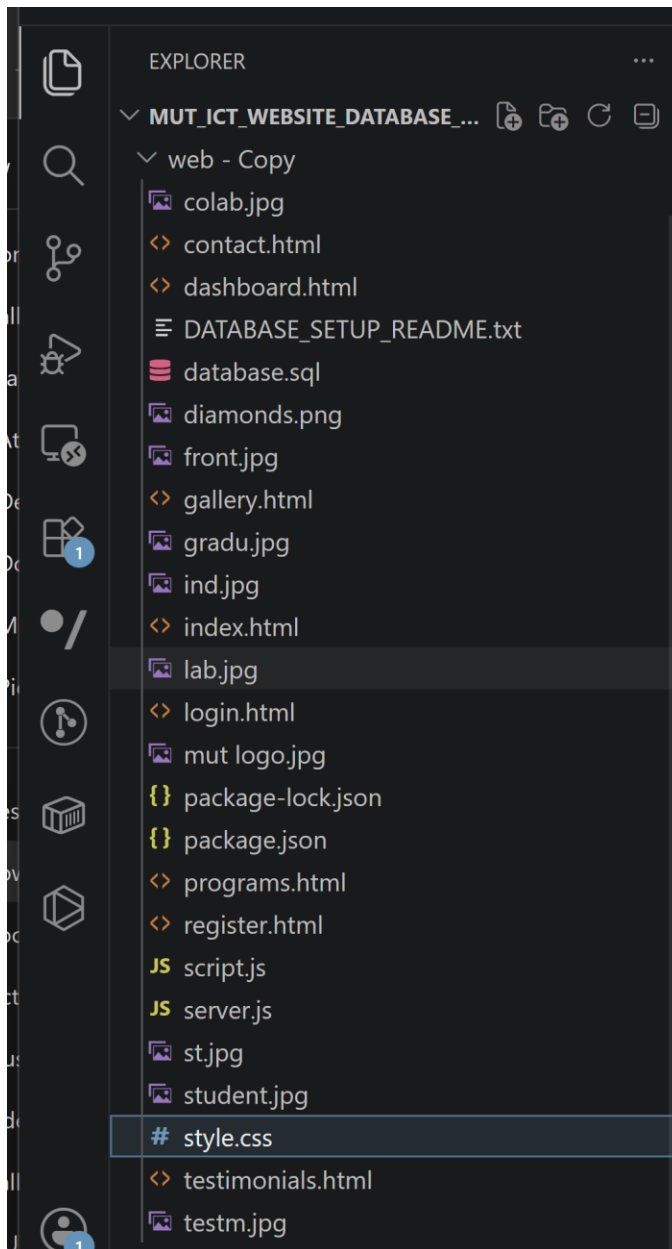
The project used an organized folder structure to improve maintainability and scalability.

The backend structure included:

- Routes folder
- Controllers folder
- Middleware folder
- Database configuration folder
- Public assets folder
- Models folder

This structure helped separate concerns and improved code organization.

PROJECT FOLDER STRUCTURE



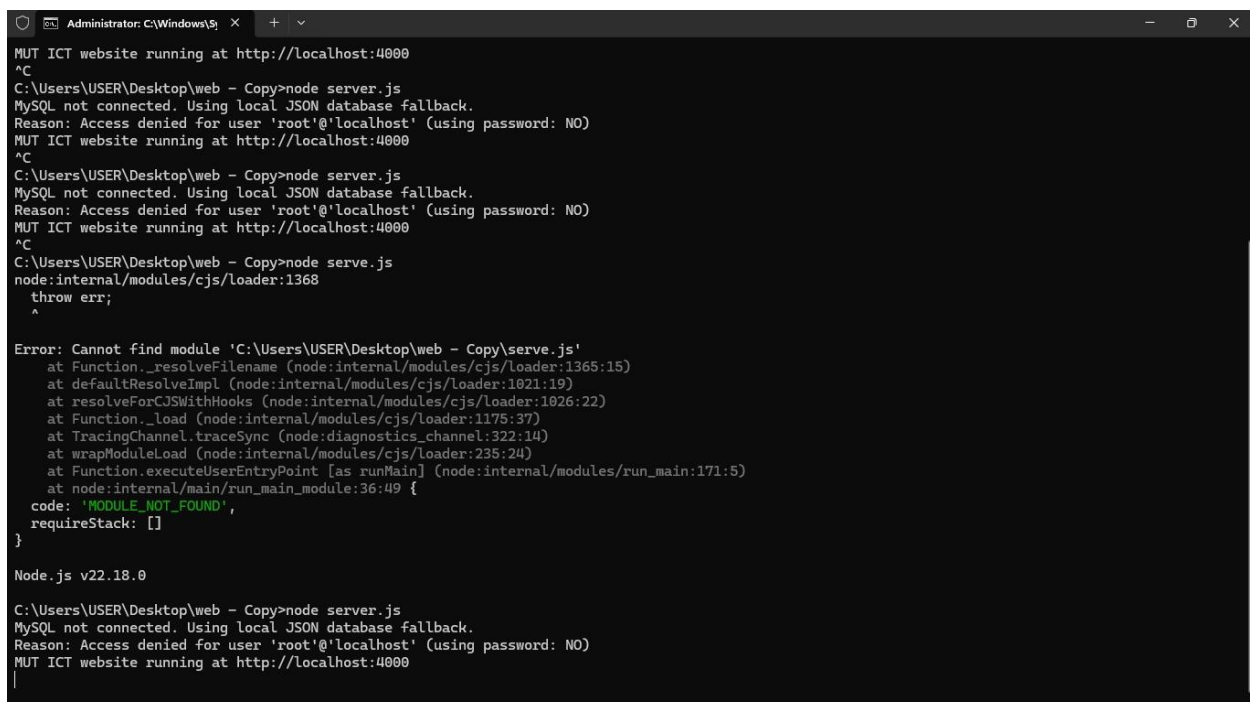
6.2 Express Server Configuration

Express was used to create the backend server. The server handled incoming HTTP requests and sent responses to the frontend.

Middleware such as body parsing, static file serving and error handling was configured within the Express application.

The server was configured to listen on a specific port and support communication between the frontend and backend.

SERVER RUNNING



```
Administrator: C:\Windows\S...
MUT ICT website running at http://localhost:4000
^C
C:\Users\USER\Desktop\web - Copy>node server.js
MySQL not connected. Using local JSON database fallback.
Reason: Access denied for user 'root'@'localhost' (using password: NO)
MUT ICT website running at http://localhost:4000
^C
C:\Users\USER\Desktop\web - Copy>node server.js
MySQL not connected. Using local JSON database fallback.
Reason: Access denied for user 'root'@'localhost' (using password: NO)
MUT ICT website running at http://localhost:4000
^C
C:\Users\USER\Desktop\web - Copy>node serve.js
node:internal/modules/cjs/loader:1368
  throw err;
  ^

Error: Cannot find module 'C:\Users\USER\Desktop\web - Copy\serve.js'
    at Function._resolveFilename (node:internal/modules/cjs/loader:1365:15)
    at defaultResolveImpl (node:internal/modules/cjs/loader:1021:19)
    at resolveForCJSWithHooks (node:internal/modules/cjs/loader:1026:22)
    at Function._load (node:internal/modules/cjs/loader:1175:37)
    at TracingChannel.traceSync (node:diagnostics_channel:322:14)
    at wrapModuleLoad (node:internal/modules/cjs/loader:235:24)
    at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:171:5)
    at node:internal/main/run_main_module:36:49 {
  code: 'MODULE_NOT_FOUND',
  requireStack: []
}

Node.js v22.18.0

C:\Users\USER\Desktop\web - Copy>node server.js
MySQL not connected. Using local JSON database fallback.
Reason: Access denied for user 'root'@'localhost' (using password: NO)
MUT ICT website running at http://localhost:4000
|
```

6.3 Routes and HTTP Methods

The system implemented multiple routes and HTTP methods to support backend operations.

The following HTTP methods were used:

- GET
- POST
- PUT/PATCH
- DELETE

The routes supported CRUD operations and interaction between users, the frontend and the database.

Route	HTTP Method	Purpose
/students	GET	Retrieve student data
/events	POST	Create event
/faq/:id	PUT	Update FAQ
/delete/:id	DELETE	Delete record
/contact	POST	Submit enquiry

7. DATABASE DESIGN AND INTEGRATION

7.1 Database Technology

The system was connected to a database to support dynamic content management and CRUD functionality.

The database stored important departmental information including:

- Student enquiries
- Staff profiles
- Events
- Announcements
- FAQs
- Feedback records

Database integration allowed the website to store and retrieve information dynamically.

DATABASE

web - Copy >  database.sql

```
1  CREATE DATABASE IF NOT EXISTS mut_website;
2  USE mut_website;
3
4  CREATE TABLE IF NOT EXISTS users (
5      id INT AUTO_INCREMENT PRIMARY KEY,
6      fullname VARCHAR(100) NOT NULL,
7      email VARCHAR(120) NOT NULL UNIQUE,
8      password VARCHAR(255) NOT NULL,
9      role VARCHAR(30) DEFAULT 'student',
10     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
11 );
12
13 CREATE TABLE IF NOT EXISTS chatbot_logs (
14     id INT AUTO_INCREMENT PRIMARY KEY,
15     user_message TEXT,
16     bot_reply TEXT,
17     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
18 );
19
```

7.2 Database Design

The database was designed to organize data efficiently and support the functionality of the application.

The database included multiple tables or collections depending on the database technology used.

Example entities included:

- Users
- Staff
- Events

- FAQs
- Enquiries
- Appointments

Relationships between entities were designed to support data consistency and efficient retrieval.

7.3 CRUD Operations

The application demonstrated CRUD operations through frontend and backend integration.

Create

Users could submit forms to add new information to the database.

Read

The system retrieved and displayed stored information dynamically.

Update

Existing records could be edited and updated.

Delete

Records could be removed from the database when necessary.

These operations demonstrated full database integration within the application.

8. MIDDLEWARE IMPLEMENTATION

Middleware was implemented to improve application functionality, security and error handling.

The middleware used in the project included:

- Logging middleware
- Validation middleware
- Error-handling middleware
- Authentication middleware

Logging middleware tracked incoming requests and assisted with debugging.

Validation middleware ensured that users entered correct data before processing requests.

Error-handling middleware improved system reliability by handling server and application errors gracefully.

Authentication middleware protected restricted routes and improved security.

Middleware improved the organization and maintainability of the Express application.

MIDDLEWARE

```
app.post('/api/register', async (req, res) => {
  try {
    const { fullname, email, password } = req.body;
    if (!fullname || !email || !password) return res.status(400).json({ success:false, message:'Enter all details.' });
    const hashed = await hashPassword(password);
    if (pool) {
      await pool.query('INSERT INTO users(fullname,email,password) VALUES (?,?)', [fullname, email, hashed]);
    } else {
      const data = readLocalUsers();
      if (data.users.some(u =&gt; u.email === email.toLowerCase())) return res.status(400).json({ success:false, message:'Email already exists.' });
      data.users.push({ id: Date.now(), fullname, email: email.toLowerCase(), password: hashed });
      writeLocalUsers(data);
    }
    res.json({ success:true, message:'Registration successful. You can now log in.' });
  } catch (err) {
    if (err.code === 'ER_DUP_ENTRY') return res.status(409).json({ success:false, message:'Email already exists.' });
    res.status(500).json({ success:false, message:'Registration failed.' });
  }
});</pre
```

ERROR HANDLING

```
154 app.post('/api/login', async (req, res) => {
155   try {
156     const { email, password } = req.body;
157     if (!email || !password) return res.status(400).json({ success:false, message:'Enter email and password.' });
158     let user;
159     if (pool) {
160       const [rows] = await pool.query('SELECT * FROM users WHERE email = ?', [email.toLowerCase()]);
161       user = rows[0];
162     } else {
163       user = readLocalUsers().users.find(u => u.email === email.toLowerCase());
164     }
165     if (!user) return res.status(401).json({ success:false, message:'User not found.' });
166     const ok = await comparePassword(password, user.password);
167     if (!ok) return res.status(401).json({ success:false, message:'Incorrect password.' });
168     req.session.user = { id:user.id, fullname:user.fullname, email:user.email, role:user.role };
169     res.json({ success:true, message:'Login successful.', user:req.session.user });
170   } catch (err) { res.status(500).json({ success:false, message:'Login failed.' }); }
171 });

172 app.get('/api/me', (req, res) => res.json({ loggedIn: !!req.session.user, user: req.session.user }));
```

9. AI AND SMART FEATURE IMPLEMENTATION

The website included a smart feature designed to improve user interaction and support.

The smart feature provided users with quick access to departmental information and assistance.

The AI-driven feature accepted user input and generated relevant responses based on predefined logic or intelligent processing.

The smart functionality improved accessibility and reduced the time required for users to locate information.

Examples of supported features included:

- Smart chatbot
- FAQ assistant
- Smart search functionality
- Automated response system

The feature improved communication between users and the ICT Department.

AI FEATURE

AI CHATBOT



10. FULL SYSTEM INTEGRATION

The final application demonstrated complete integration between the frontend, backend, database and middleware.

Users interacted with the frontend interface through forms, navigation systems and smart features. The frontend sent requests to the backend server using HTTP methods.

The backend processed requests, interacted with the database and returned appropriate responses to the frontend.

Middleware components validated requests, logged activity and handled errors before responses were returned.

This integration created a complete full-stack web application.

11. TESTING AND RESULTS

System testing was conducted to ensure that all components worked correctly.

Testing focused on:

- Frontend functionality
- Form validation
- Route testing
- Database operations
- Middleware processing
- Responsiveness
- AI feature functionality

The testing results showed that the system operated successfully with minimal errors.

Feature	Expected Result	Actual Result	Status
Login System	User logs in successfully	Successful	Pass
Form Validation	Invalid input rejected	Successful	Pass

Database Connection	Data stored successfully	Successful	Pass
CRUD Operations	Records managed correctly	Successful	Pass
AI Feature	User receives responses	Successful	Pass

12. CHALLENGES FACED

Several challenges were experienced during the development of the project.

One challenge involved configuring the database connection correctly and ensuring stable communication between the backend and database.

Another challenge involved debugging Express routes and handling asynchronous operations.

The group also experienced challenges related to responsive design and ensuring consistent layouts across devices.

Middleware integration and error handling required additional testing to ensure that invalid requests were processed correctly.

These challenges were solved through debugging, research, testing and collaboration among group members.

13. CONCLUSION

The project successfully transformed the ICT Department website into a smart, interactive and database-driven web application.

The redesigned system improved usability, responsiveness, accessibility and user engagement through the implementation of UXD principles and interactive functionality.

The integration of Node.js, Express, database connectivity, middleware and AI functionality demonstrated the successful development of a full-stack web application.

The project also demonstrated understanding of frontend development, backend processing, database integration, middleware implementation and smart website functionality.

Overall, the system achieved the objectives of improving the departmental website and providing a more modern and intelligent user experience.